UNIVERSIDAD EIA
Ser, Saber y Servir
Con Acreditación Institucional

Sistemas de Control en Tiempo Continuo · Ingeniería Mecatrónica · revisión 2026-2

Manual de funcionamiento y uso de la MPA

Módulo de posición angular: conexión, enlace con Simulink, calibración, ensayos e identificación de un modelo en espacio de estados

Planta	Lámina de acrílico que cuelga de un eje y que el chorro de una hélice levanta. Entrada: mando u [%]. Salida: ángulo θ [°], medido con un potenciómetro.
Enlace	Firmware propio en el Arduino y bloques seriales en Simulink. El Arduino marca el reloj (50 Hz), aplica el mando al ESC con rampa y tope, y vigila la comunicación.
Software	MATLAB y Simulink con el *Instrument Control Toolbox* (bloques *Serial*). Arduino IDE para cargar el firmware una vez.
Qué sale de aquí	La **ficha de su módulo** (calibración, límites del mando, mapa estático) y un **modelo lineal** $\dot{x} = Ax + Bu$, $y = Cx$ alrededor de un punto de operación, con estados físicos (ángulo y velocidad angular).
Qué no está aquí	El diseño del controlador y del observador: eso es el Reto In-Situ 3. Este manual deja la planta, el modelo y la interfaz de lazo cerrado listos para él.

Índice

1 Antes de empezar 1
2 La planta y por qué se comporta como un péndulo 2
3 Conexiones y seguridad 4
4 La hélice: montaje y sentido de giro 5
5 El enlace con Simulink 7
6 El sensor: de cuentas a grados 11
7 El mapa estático 14
8 Identificación para espacio de estados 17
9 Preparar el lazo cerrado 24
10 Problemas frecuentes 25
11 Lista de comprobación 26
A Ficha del módulo 27
B Hojas de campo 28
Procedencia 28

1. Antes de empezar

El laboratorio tiene seis MPA. Todas son el mismo diseño, pero no son idénticas: han sido reparadas en momentos distintos, los potenciómetros y los ESC no son todos iguales y cada lámina tiene su propio rozamiento. Por eso **nada de lo que se mide aquí se copia de otra unidad**. Cada módulo tiene una **ficha** (anexo A) donde quedan sus datos; si ya está llena, la primera tarea es comprobar tres de sus valores. En el texto, la marca **[ficha]** indica un dato que va a la ficha.

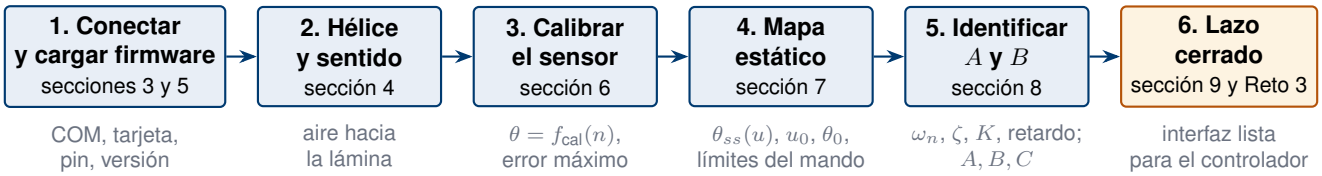
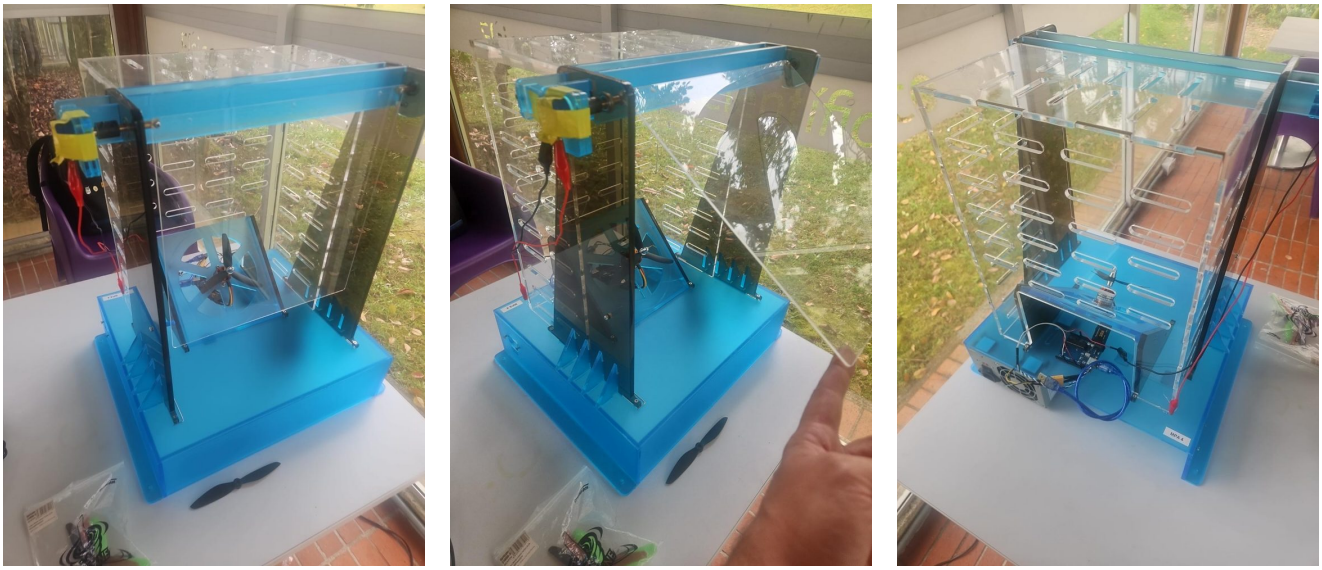


Figura 1: Ruta de trabajo. Debajo de cada etapa, lo que queda en la ficha.

Reglas de trabajo

- **Un archivo de Simulink por etapa**, guardado con el número del módulo (`MPA_XX_ensayos.slx`, `MPA_XX_lazo.slx`). No reutilicen el de otra pareja: arrastra su calibración.
- **Todo dato se guarda con su contexto**: módulo, fecha, punto de operación y amplitud del ensayo.
- **Nunca abran un modelo de la MPA en Connected IO ni en Run on board**: Simulink reemplazaría el firmware del Arduino por el suyo y habría que volver a cargarlo (sección 5.2).

2. La planta y por qué se comporta como un péndulo



(a) vista frontal (b) la lámina gira sobre el eje superior (c) vista posterior

Figura 2: La MPA 4. (a) La hélice, dentro de su protección inclinada, sopla hacia la lámina transparente; arriba a la izquierda, el potenciómetro (cinta amarilla) acoplado al eje. (b) La lámina cuelga del eje superior y el chorro la levanta. (c) Detrás: Arduino, ESC, motor y fuente ATX.

Una lámina de acrílico cuelga de un eje horizontal. Un motor *brushless* mueve una hélice cuyo chorro empuja la lámina hacia arriba; la gravedad la devuelve y el aire y los rodamientos la frenan. Con el motor apagado, la lámina descansa en 0° contra su tope. Un potenciómetro acoplado al eje entrega una tensión que depende del ángulo.

2.1 El modelo físico

Visto desde el eje, es un péndulo con un empuje que depende de la velocidad de la hélice ω_h y del propio ángulo, porque al subir la lámina cambia cómo la golpea el chorro:

$$J\,\ddot{\theta} + b\,\dot{\theta} + \tau_g(\theta) = \tau_a(\theta, \omega_h), \qquad \tau_m\,\dot{\omega}_h = -\omega_h + g_m(u). \tag{1}$$

J es la inercia de la lámina, b el amortiguamiento (aire y rodamientos), τ_g el par de la gravedad, τ_a el par del chorro y τ_m la constante de tiempo del conjunto ESC–motor–hélice. Con el ángulo medido desde la vertical de reposo, $\tau_g = mgd\sin\theta$, y su pendiente $mgd\cos\theta$ se va a cero cerca de 90°: allá arriba la gravedad casi no devuelve la lámina.

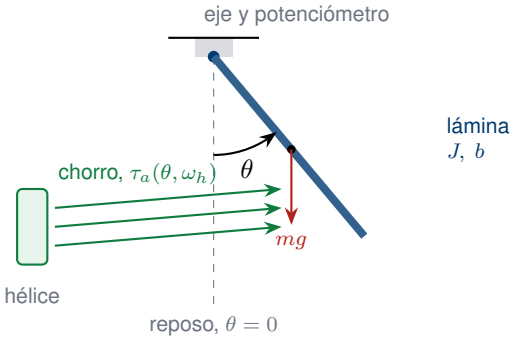


Figura 3: Esquema (no a escala). El chorro levanta la lámina, la gravedad la devuelve. El equilibrio aparece donde los dos pares se igualan.

2.2 Por qué el modelo depende del punto de operación

Alrededor de un equilibrio (u_0, θ_0), con desviaciones $\tilde{\theta} = \theta - \theta_0$ y $\tilde{u} = u - u_0$, la ecuación (1) se linealiza en

$$J\,\ddot{\tilde{\theta}} + b\,\dot{\tilde{\theta}} + k\,\tilde{\theta} = \kappa\,\tilde{u}, \qquad k = \left.\frac{\partial \tau_g}{\partial \theta} - \frac{\partial \tau_a}{\partial \theta}\right|_0, \tag{2}$$

suponiendo que la hélice responde mucho más rápido que la lámina (τ_m pequeño). La rigidez k y la ganancia κ **cambian con** θ_0: un modelo identificado cerca de 40° describe la planta cerca de 40°, no a 70°.

Dos no linealidades que no se deben confundir

La curvatura del potenciómetro afecta la *medición*; el cambio de k y κ con el ángulo afecta la *planta*. La calibración corrige la primera. La segunda no desaparece: se maneja escogiendo un punto de operación y, si el reto lo pide, identificando más de uno.

2.3 Señales y magnitudes

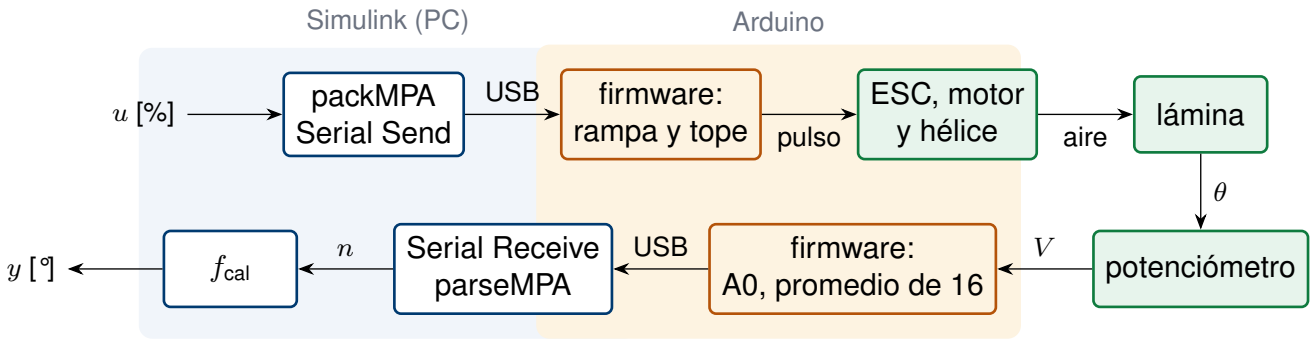


Figura 4: Cadena completa de actuación y medida.

Cuadro 1: Magnitudes que no deben confundirse.

Símbolo	Unidad	Significado
u	%	Mando que pide Simulink (0–100). Se convierte en un ancho de pulso para el ESC (sección 5.4).
u_{ap}	%	Mando que el Arduino aplicó de verdad, después de la rampa y el tope. Lo reporta en cada paquete.
p	µs	Ancho del pulso que recibe el ESC, cada $20\,\text{ms}$.
n	cuentas	Lectura del ADC en A0 (0–1023), promedio de 16 lecturas. Sale de `parseMPA` como `adc`.
θ, y	°	Ángulo físico de la lámina y su estimación a partir de n con la calibración.
u_0, θ_0	%, °	Punto de operación: mando constante y ángulo de equilibrio que produce.
$\tilde{u}$, $\tilde{\theta}$	%, °	Desviaciones respecto al punto de operación. El modelo lineal trabaja con ellas.

3. Conexiones y seguridad

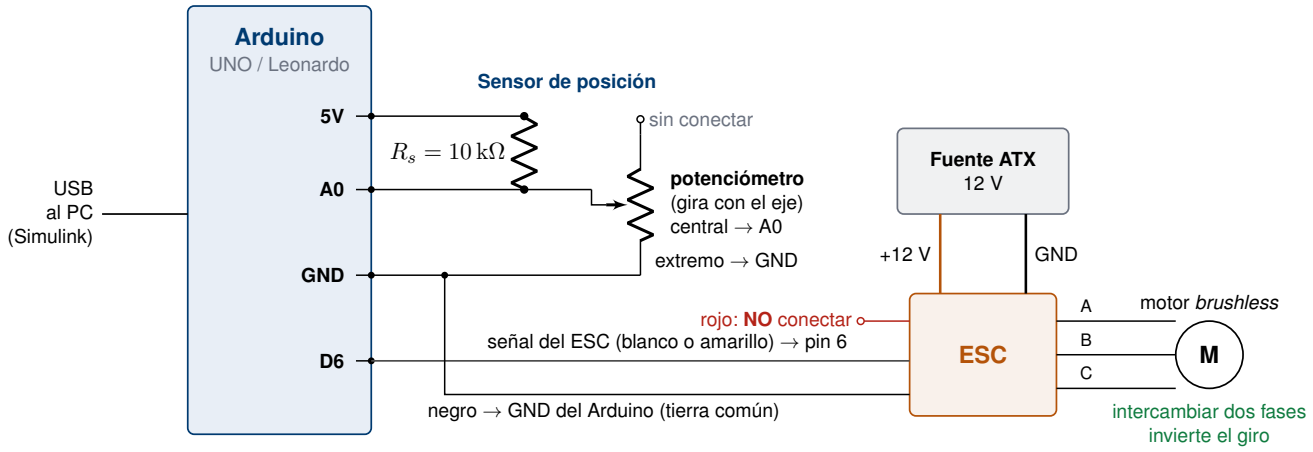


Figura 5: Conexiones de la MPA.

- **Sensor**: el terminal central del potenciómetro va a **A0**; de A0 sale una resistencia de 10 kΩ hacia **5 V**; un extremo del potenciómetro va a **GND** y el otro queda libre. A0 ve el divisor $V_{A0} = 5\,\text{V} \cdot$

$R_{\text{pot}}(\theta)/(R_{\text{pot}}(\theta)+R_s)$, que **no es lineal** en θ: por eso se calibra (sección 6).

- **Señal del ESC**: del cable de tres hilos del ESC, el de **señal** (blanco o amarillo) va al **pin 6** y el **negro** a GND del Arduino. Sin esa tierra común el ESC no entiende la señal. El **rojo no se conecta**. **[ficha]**
- **Potencia**: el ESC se alimenta de los 12 V de la fuente ATX (amarillo) y su GND (negro). El Arduino se alimenta y se comunica por USB.
- **Motor**: el ESC sale al motor por tres cables. Su orden solo decide el sentido de giro; **intercambiar dos cualesquiera lo invierte** (sección 4).

Seguridad: antes de cada ensayo

- Manos, cabello, cables y objetos lejos de la hélice y del recorrido de la lámina. **Nunca** se toca la lámina ni la hélice con la fuente encendida.
- La fuente se enciende con el modelo de Simulink ya corriendo y $u = 0$; se apaga antes de detenerlo.
- Si el motor se calienta, vibra de forma anormal o la fuente se protege, paren el ensayo. Una fuente ATX protegida no vuelve sola: hay que desconectarla de la red unos segundos y volver a conectarla.
- Revisen que la lámina no esté doblada y que los tornillos estén firmes. Una lámina doblada cambia el modelo.

4. La hélice: montaje y sentido de giro



(a) cara con el tamaño (6×4.5) (b) cara opuesta (marca) (c) montada en la MPA 4 (d) detalle del montaje

Figura 6: Hélice 6×4.5 y su montaje en la MPA 4, que funcionó después de corregir la orientación y el sentido de giro.

Una hélice no es simétrica: tiene una cara que empuja el aire y un sentido de giro para el que está diseñada (algunas lo indican con una flecha o marcas en el cubo). Con la hélice al revés o girando al

revés, el motor gira, hace ruido y la lámina apenas se mueve. Es el error más difícil de ver y el que más tiempo hace perder.

La regla: el aire tiene que ir hacia la lámina

Piensen en un dron: para despegar, sus hélices empujan el aire hacia el **piso**. En la MPA el «piso» es la **lámina**: la hélice se monta de manera que la cara que en un dron miraría al piso mire hacia la lámina. Si una hélice venía montada al contrario, la planta no levanta la lámina aunque todo lo demás esté bien.

Verificación del sentido del aire (cada vez que se cambie o se desmonte la hélice)

1. Monten la hélice con la orientación de la regla anterior y aprieten la tuerca.
2. Con el modelo corriendo y el motor armado (sección 5), lleven u a un valor bajo, **alrededor de 20 %**.
3. Comprueben hacia dónde va el aire con una tira de papel sostenida **lejos** de la hélice, entre la protección y la lámina. Nunca con la mano.
4. Si el aire **no** va hacia la lámina: $u = 0$, apaguen la fuente, **intercambien dos de los tres cables** entre el ESC y el motor y repitan.
5. Con el aire hacia la lámina, la lámina debe empezar a despegarse con un mando bajo (en la MPA 4, cerca de $u = 10\,\%$). **[ficha]**

Por qué funciona este orden

Con la hélice bien orientada, de los dos sentidos de giro solo uno manda el aire hacia la lámina, y es justamente el sentido para el que la hélice está diseñada. Si la hélice estuviera volteada, el giro que manda aire hacia la lámina sería el «equivocado» y el empuje sería pobre. Por eso primero se fija la orientación y después se corrige el giro con los cables.

5. El enlace con Simulink

Simulink no maneja la MPA directamente. Un **firmware propio** en el Arduino hace el trabajo de tiempo real (medir, aplicar el pulso al ESC, proteger) y conversa con Simulink por el USB con paquetes binarios. Simulink calcula el controlador y envía el mando. No hace falta entender cada línea del código; sí hace falta saber qué hace, porque dos de sus funciones (la rampa y el tope) son parte del actuador que se va a modelar.

5.1 Qué hace el firmware

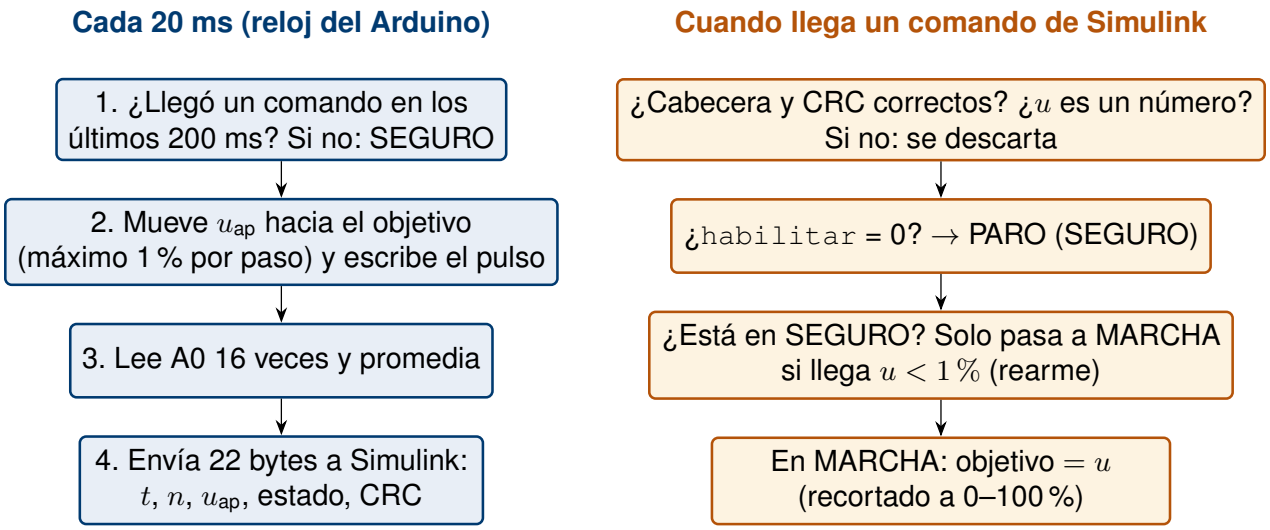


Figura 7: Las dos tareas del firmware. El estado SEGURO lleva el mando a 0 (en rampa) y espera un rearme.

Cuadro 2: Parámetros del firmware (al inicio de `MPA_Firmware.ino`). Los que cambian por unidad van a la ficha.

Parámetro	Valor	Qué hace
`PIN_ESC`	6	Pin de la señal del ESC. **[ficha]**
`PERIODO_US`	20000	Periodo del lazo: $20\,\text{ms}$ = 50 Hz. Es el T_s de Simulink.
`PULSO_0`	1000 µs	Pulso con $u = 0$: motor detenido y pulso de armado del ESC.
`PULSO_100`	1450 µs	Pulso con $u = 100\,\%$: máxima velocidad útil. **[ficha]**
`PULSO_TOPE`	1450 µs	Ningún pulso lo supera, aunque Simulink pida más. **[ficha]**
`RAMPA_PCT_S`	50 %/s	Rapidez máxima de cambio del mando aplicado (1 % por paso).
`TIMEOUT_MS`	200	Sin comandos válidos por ese tiempo, el mando baja a 0.
`U_REARME`	1 %	Después de SEGURO, el motor solo arranca si llega u menor que esto.
`N_ADC`	16	Lecturas de A0 promediadas por muestra.

5.2 Cargar el firmware

El archivo está en el repositorio del curso, carpeta `firmware/MPA_Firmware/`. La carpeta y el archivo `.ino` deben llamarse igual, como exige el Arduino IDE.

Grabar el firmware (una vez por unidad, o cuando alguien lo haya borrado)

1. Fuente del ESC **apagada**. Conecten el Arduino por USB.
2. Abran `MPA_Firmware.ino` con el Arduino IDE (2.x o 1.8.x). No hace falta instalar librerías: `Servo` viene con el IDE.
3. *Tools* → *Board*: **Arduino Uno** o **Arduino Leonardo**, según su unidad. *Tools* → *Port*: el COM del Arduino. **[ficha]**
4. Revisen al inicio del archivo `PIN_ESC` (6) y, si la ficha lo indica, `PULSO_100` y `PULSO_TOPE`.
5. *Upload* (flecha →). Debe terminar con «Done uploading». Si el puerto está ocupado, cierren MATLAB y el monitor serie.
6. **Cierren el monitor serie del IDE** antes de abrir Simulink: si queda abierto, ocupa el puerto. (Si lo abren, verán símbolos raros: los paquetes son binarios, no texto.)
7. Peguen en la unidad el rótulo «Firmware MPA v2: no usar Connected IO».

¿Cómo sé que el firmware está bien?

Con el modelo de Simulink corriendo y la fuente apagada: `valido` $= 1$, `estado` pasa a 1 con $u = 0$, `diff(t_ard)` $= 0.02$ y `adc` cambia al mover la lámina con la mano. Si Simulink se queda esperando y da *timeout*, el Arduino no tiene este firmware o el puerto es otro.

5.3 El subsistema MPA en Simulink

Todo el enlace se encapsula en un subsistema **MPA**: entra el mando, salen las medidas. Es el mismo esquema que se usó con la bomba de infusión de Biomecatrónica.

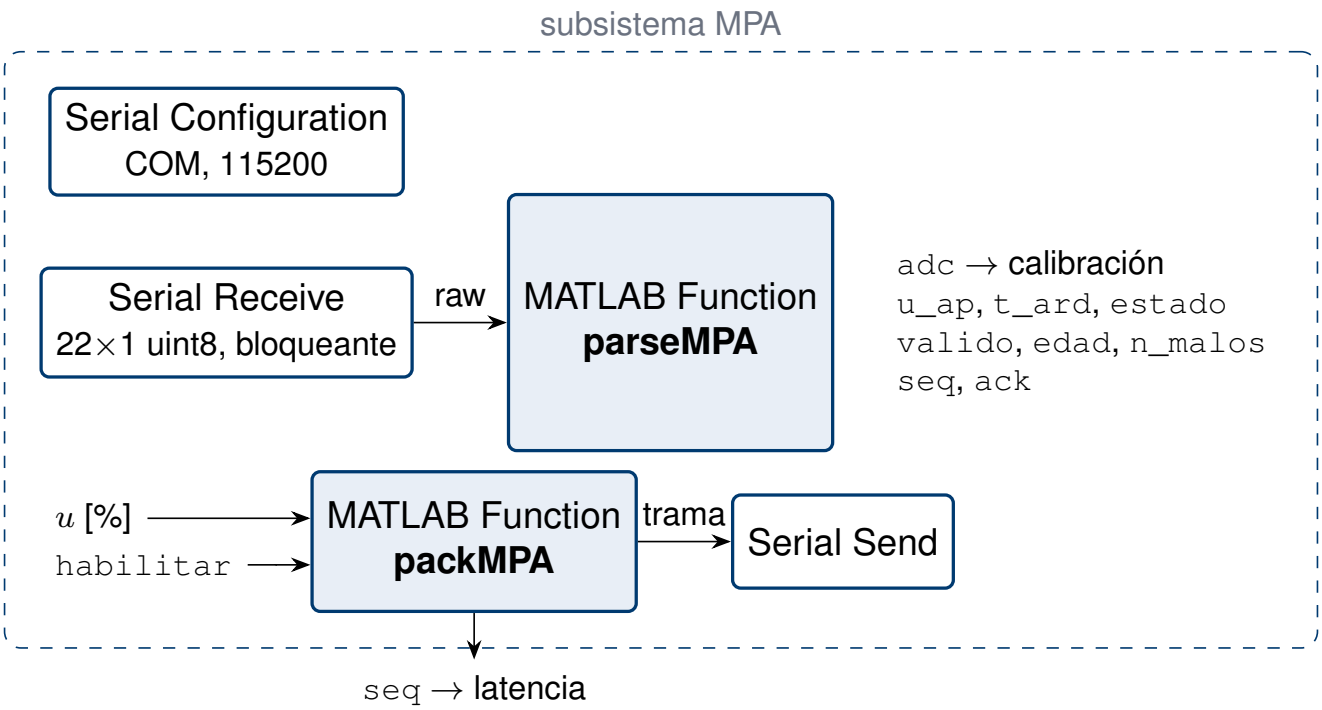


Figura 8: Subsistema **MPA**. Los bloques seriales son del *Instrument Control Toolbox*; el código de las dos MATLAB Function está en los archivos `parseMPA.m` y `packMPA.m`.

Cuadro 3: Configuración de los bloques y del modelo.

Dónde	Valor
Serial Configuration	Puerto COM de la unidad, 115200, 8 bits, sin paridad, 1 de parada, sin control de flujo, *Timeout* 5 s. **[ficha]**
Serial Receive	*Header* vacío, *Terminator* `<none>`, *Data size* `[22 1]`, *Data type* `uint8`, ***Enable blocking mode*** **activado**, *Block sample time* 0.02.
Serial Send	*Header* vacío, *Terminator* `<none>`.
MATLAB Function	Pegar el contenido *completo* de cada `.m`, con sus funciones auxiliares del final. En *Edit Data*, las salidas de `parseMPA` con tipo *Inherit* o `double`: si `u_ap` queda en `uint8`, pierde los decimales y el lazo del Reto 3 no puede restarla de un `double`.
Solver	Paso fijo, `ode4` (o `ode1`), $T_s = 0.02$ s. No `discrete`: el controlador y el observador del Reto 3 son bloques continuos. Stop time según el ensayo.
Simulation Pacing	**Apagado**: el Arduino marca el paso (ver abajo).
Hardware board	`None`.
Data Import/Export	*Single simulation output* **desmarcado** y *Time* (`tout`) marcado: cada **To Workspace** deja su variable suelta en el workspace, sin `out`.

Por qué el tiempo de Simulink es tiempo real

El **Serial Receive** bloqueante espera a que llegue cada paquete, y el Arduino envía uno cada $20\,\mathrm{ms}$ con su propio reloj. Simulink avanza entonces exactamente un paso de $0.02\,\mathrm{s}$ por paquete: el tiempo de simulación *es* el tiempo del Arduino. Si el PC se retrasa, los paquetes esperan en el puerto y Simulink los consume más rápido hasta ponerse al día. Con *Simulation Pacing* activado ese atraso no se recuperaría nunca.

5.4 La escala del mando

El ESC recibe la señal estándar de servo: un pulso cada $20\,\mathrm{ms}$, de $1\,\mathrm{ms}$ (5 % del periodo) a $2\,\mathrm{ms}$ (10 %). En la MPA el motor llega a su máxima velocidad útil mucho antes de $2\,\mathrm{ms}$, así que el firmware reparte el 0–100 % solo en la franja útil:

$$p = 1000 + \frac{u}{100}\,(1450 - 1000)\ \mu\mathrm{s} = 1000 + 4.5\,u\ \mu\mathrm{s}. \tag{3}$$

Cuadro 4: Escala del mando en la MPA 4 (comprueben la suya).

u [%]	p [µs]	duty [%]	Qué pasa
0	1000	5.0	Motor detenido; pulso de armado.
≈7	≈1030	5.2	Empieza a girar la hélice (u_{arr}).
≈10	≈1045	5.2	La lámina se despega del tope (u_{mov}).
100	1450	7.3	Máxima velocidad útil; la lámina pasa de 90°.

5.5 Rampa y tope: dos no linealidades del actuador

- **Rampa**. El mando aplicado cambia como máximo 50 %/s (1 % por paso). Sin ella, un cambio brusco del mando hacía que el motor pidiera un pico de corriente al acelerar o devolviera energía a los 12 V al frenar, y la fuente ATX se protegía. La rampa es un **limitador de rapidez** ($|\dot{u}_{\mathrm{ap}}| \le 50\,\%/\mathrm{s}$): una no linealidad que solo actúa cuando se pide un cambio grande y rápido. Para cambios pequeños alrededor del punto de operación no se activa, y el modelo lineal sigue valiendo.
- **Tope**. Ningún pulso supera `PULSO_TOPE`; junto con el recorte de u a 0–100 % es una **saturación**.

Qué consecuencias tiene para el control

- El lazo simulado lleva un **Rate Limiter** de $\pm 50\,\%/\mathrm{s}$ y una **Saturation** antes del modelo de la planta; si no, lo simulado y lo real se separan en los cambios grandes.
- El Arduino reporta u_{ap}, el mando que de verdad recibió el ESC después de la rampa y del tope. Esa es la señal que un controlador con estados internos debe seguir (sección 9).
- En la identificación, el pulso de mando sale «redondeado» por la rampa. Por eso el análisis empieza cuando la rampa terminó de bajar (sección 8.3).

5.6 Armado, rearme y PARO

Secuencia de arranque (siempre la misma)

1. Fuente **apagada**. Modelo con $u = 0$ y `habilitar` $= 1$. *Run*.
2. Comprueben en Simulink: `estado` $= 1$, `valido` $= 1$, latencia 0 o 1.
3. **Ahora** enciendan la fuente: suena la secuencia corta de armado y el ESC queda en silencio. Desde que el Arduino arranca envía el pulso de $1000\,\mu\mathrm{s}$, así que el ESC arma siempre que la fuente se encienda con el Arduino ya funcionando.
4. Para terminar: $u = 0$, esperen a que la hélice pare, apaguen la fuente y detengan Simulink.

- **PARO**: `habilitar` $= 0$ (un **Manual Switch** entre dos constantes) lleva el mando a 0 en rampa.

- **Rearme**: después de un PARO, de un corte de comunicación o del arranque, el motor solo vuelve a moverse cuando llega un $u < 1\,\%$. Si el modelo arranca con $u \neq 0$, el motor no se mueve y `estado` se queda en 0: no es una falla.
- **Corte de comunicación**: si Simulink se detiene o el cable se suelta, a los $200\,\mathrm{ms}$ el mando baja solo a 0.

5.7 Diagnóstico del enlace

- **Latencia** $= \mathrm{mod}(\mathrm{seq}_{k-1} - \mathrm{ack}_k,\, 65536)$: el número del comando anterior (salida `seq` de `packMPA` por un **Unit Delay**) menos el último que el Arduino aceptó (`ack`). Debe ser 0 o 1. Si crece, el PC no alcanza los 50 Hz: cierren Scopes y quiten Displays.
- `diff(t_ard)` debe valer 0.02 en todos los pasos; `valido` en 1; `n_malos` en 0 o casi.

6. El sensor: de cuentas a grados



La salida `adc` de `parseMPA` ya está en cuentas (promedio de 16 lecturas, tipo *double*), así que se calibra **directamente de cuentas a grados**, $\theta = f_{\mathrm{cal}}(n)$: no hace falta la ganancia de voltios ni un **Data Type Conversion**. La relación no es lineal (sección 3) y puede subir o bajar al levantar la lámina según cómo esté montado el potenciómetro. Varias unidades tienen potenciómetros de reemplazo: miren la referencia impresa en el suyo. **[ficha]**

Figura 9: Potenciómetro multivuelta típico (ilustración).

6.1 Calibración con el motor apagado

Procedimiento

1. Fuente del ESC **apagada**. Modelo con el subsistema **MPA**, `adc` a un **Display** y a un **To Workspace**. *Run*.
2. Muevan la lámina con la mano despacio: la lectura debe cambiar en forma suave y repetible.
3. Lleven la lámina a 0, 10, …, 90° según el indicador, **subiendo**. Esperen unos segundos en cada marca y anoten `adc`. Repitan **bajando** (hoja de campo, anexo B).
4. Anoten además tres marcas **que no van a usar para ajustar** (15, 45 y 75°): son para validar.

Medir el ángulo con el celular

Si el indicador de su módulo está borroso, flojo o no tiene todas las marcas, usen una app de **nivel o inclinómetro** en el celular. En iPhone, la app *Medidas* trae un nivel; en Android sirve cualquier app de nivel de burbuja o clinómetro.

1. Calibren la app sobre una mesa horizontal, si la app lo permite.
2. Con la lámina **colgando en reposo**, apoyen el celular de canto contra su cara plana y anoten

la lectura: ese es el 0° de su lámina. Lo que importa es el cambio desde ahí, no el número absoluto.

3. Para cada marca, sostengan la lámina con una mano y el celular contra ella con la otra, sin empujar el eje. Lean el ángulo cuando la lectura esté quieta y anoten `adc` al mismo tiempo.
4. Usen siempre la misma cara de la lámina y la misma orientación del celular.

Con esto la incertidumbre es de 1° aproximadamente, del orden del error de ajuste que se acepta. Si el indicador y la app no coinciden en más de 2°, confíen en la app y avisen al laboratorio.

6.2 El ajuste en MATLAB

El guion `calibracion_sensor.m` (carpeta `scripts/` del repositorio) hace todo el ajuste: solo hay que copiar las lecturas de la hoja de campo en su sección `DATOS` y ejecutarlo (F5).

Código 1: La única parte que se edita en `calibracion_sensor.m`.

```
%% DATOS (llenar con SU modulo)
modulo  = 4;                    % numero de la MPA
theta   = 0:10:90;              % marcas del indicador [grados]
n_sub   = [ ];                  % adc subiendo   (10 valores, uno por marca)
n_baj   = [ ];                  % adc bajando    (10 valores, uno por marca)
theta_v = [15 45 75];           % marcas de validacion (NO se usan en el ajuste)
n_v     = [ ];                  % adc en esas marcas (3 valores; puede quedar vacio)
```

El guion:

1. revisa que la lectura sea monótona y que la histéresis no pase de 5 cuentas;
2. ajusta una recta y una cuadrática, y **escoge la recta si su error máximo es menor a 1°**; si no, la cuadrática;
3. calcula el error en las marcas de validación, grafica los ajustes y el error en cada marca;
4. escribe los coeficientes como $y = a_2\,n^2 + a_1\,n + a_0$ (con $a_2 = 0$ si es lineal) y los guarda, con el error, la histéresis, la resolución y la fecha, en `cal_MPAxx.mat`. **[ficha]**

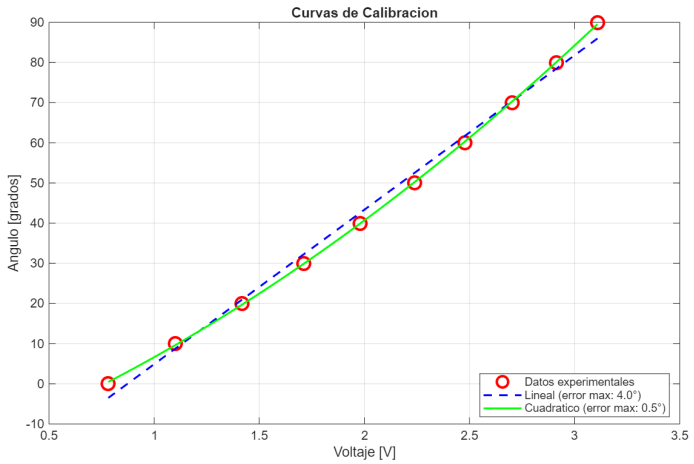


Figura 10: Ejemplo de la guía de 2026-1 (en voltios, una unidad): la recta deja errores de 4° y la cuadrática de 0.5°. En cuentas la forma es la misma; los coeficientes son los de su módulo.

6.3 Llevar la calibración a Simulink

Los coeficientes no se copian a mano en los bloques: los bloques usan los *nombres* `a2`, `a1` y `a0`, y el modelo carga el archivo `cal_MPAxx.mat` cada vez que arranca.

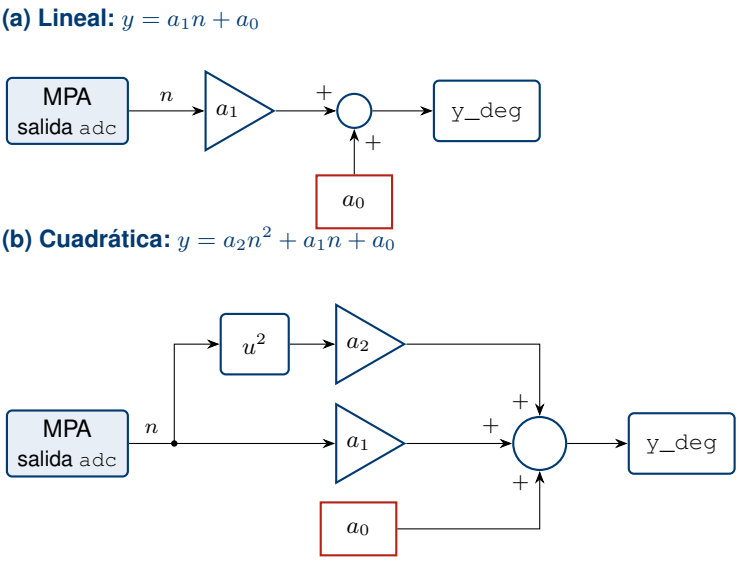


Figura 11: Bloques de la calibración, a la salida `adc` del subsistema **MPA**. Ganancias en triángulo (**Gain**), constante en rojo (**Constant**), sumador **Sum** con signos `++` o `+++`, y **Math Function** en `square` para u^2. En el campo *Gain* o *Constant value* se escribe el *nombre* (`a1`, `a2`, `a0`), no el número. La forma (b) sirve para los dos casos: si el ajuste es lineal, `a2` vale 0.

Para que el modelo cargue la calibración solo

1. Copien `cal_MPAxx.mat` en la misma carpeta del modelo `.slx`.
2. En Simulink: pestaña *Modeling* → flecha de *Model Settings* → *Model Properties* → *Callbacks* → `InitFcn`. Escriban:
   `load('cal_MPA04.mat')` (con el número de su módulo)
3. *OK* y guarden el modelo. Desde ahora, cada *Run* toma los coeficientes de ese archivo; si recalibran, basta volver a correr el guion.
4. Va en `InitFcn`, **no** en `PreLoadFcn`: esta solo corre al abrir el modelo, y cualquier `clear` posterior borra `a2`, `a1` y `a0`. El *Run* falla entonces porque los bloques no encuentran esas variables.

Comprobación y alternativas

- Con la fuente apagada, lleven la lámina a 0°, a una marca intermedia y cerca de 90°: `y_deg` debe coincidir con el indicador dentro del error de validación.
- Equivalentes: un **Fcn** con la expresión `a2*u^2 + a1*u + a0`, o una **1-D Lookup Table** con las cuentas en orden creciente como *breakpoints* y los ángulos como *table data*.
- No usen grado 3 o mayor: entre marcas oscila. Si la lectura no es monótona, el problema es el acople, no el polinomio: avisen al laboratorio.

7. El mapa estático

Antes de hablar de dinámica hay que saber **dónde se queda la lámina para cada mando**. El mapa estático $\theta_{ss}(u)$ dice dónde empieza a actuar la planta, qué mando da el ángulo que se quiere, cuánta histéresis hay y cuánto vale la ganancia local en cada zona.

7.1 Los tres valores de mando

Cuadro 5: Primera prueba con la hélice puesta: subir u de 2 en 2 cada 5 s y anotar.

Dato	Para qué sirve
u_{arr}**: gira la hélice**	**Límite inferior de la saturación** en lazo cerrado. Por debajo el motor no hace nada; si la saturación queda en 0, el controlador cree que actúa cuando no actúa y su anti-windup no se entera. **[ficha]**
u_{mov}**: se despega la lámina**	**Dónde empieza la planta**. Entre u_{arr} y u_{mov} la lámina sigue apoyada: es una zona muerta de la planta. El mapa empieza un poco por debajo. **[ficha]**
u_{85}**: lámina en ≈85°**	**Dónde termina el mapa**. Si con $u = 100\,\%$ la lámina golpea arriba, este es también el límite superior u_{sup} del lazo. **[ficha]**

7.2 El modelo de ensayos

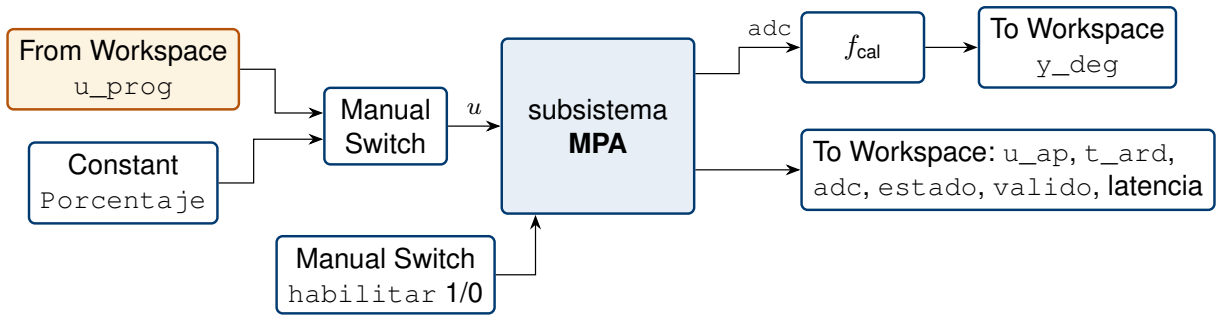


Figura 12: `MPA_XX_ensayos.slx`. El mando sale de un programa preparado en MATLAB o, con el **Manual Switch**, de una constante para la primera prueba. **From Workspace**: *Interpolate data* desmarcado, *Sample time* 0.02, *Holding final value*. Todos los **To Workspace** en formato *Array* con *Sample time* -1: cada variable queda en el workspace como un vector columna, con un valor cada 0.02 s.

7.3 El ensayo de escalera

El mando sube por escalones desde un poco por debajo de u_{mov} hasta u_{85} y vuelve a bajar. Cada escalón dura $T_h = 10\,\text{s}$: unos 6 s para que la lámina se asiente y los últimos 3 s (`Tprom`) para promediar. Si al final del escalón la lámina todavía oscila, usen 15 s.

El ensayo usa dos guiones, como la identificación: `programa_escalera.m` antes del *Run* y `analisis_escalera` después.

Código 2: `programa_escalera.m`: lo que se ajusta antes del *Run*.

```
modulo = 4;                  % numero de SU MPA
modelo = 'MPA_04_ensayos';   % nombre del .slx (sin extension)
uini = 6;     % [%]  primer escalon: un poco por debajo de u_mov (primera prueba)
ufin = 90;    % [%]  ultimo escalon: u con la lamina cerca de 85 grados
du   = 6;     % [%]  tamano del escalon (4 si quieren mas detalle)
Th   = 10;    % [s]  duracion de cada escalon (15 s si la lamina sigue oscilando)
Tprom = 3;    % [s]  ventana final de cada escalon que se promedia (ya asentada)
Tarm = 20;    % [s]  u = 0 al inicio: ENCENDER LA FUENTE en estos segundos
```

1. **`programa_escalera.m` (F5, antes del ensayo)**: borra los datos de una corrida anterior, arma el programa de mando `u_prog` y, si el modelo está abierto, fija el *Stop time* y deja el **From Workspace** sin interpolar. Con *Interpolate data* marcado los escalones se vuelven una rampa continua y la lámina nunca se asienta. Con los valores del ejemplo son 29 escalones, unos 5 minutos.
2. *Run* en Simulink con el **Manual Switch** en el programa; enciendan la fuente durante los primeros 20 s (con $u = 0$).
3. **`analisis_escalera.m` (F5, después del ensayo)**, con las variables del ensayo en el workspace:
   - revisa el enlace: `diff(t_ard)`, la latencia y que el tiempo del Arduino avance igual que el de Simulink (si Simulink va más lento, cada paso consumió más de un paquete);
   - avisa si el mando no fue constante dentro de los escalones;
   - promedia los últimos `Tprom` segundos de cada escalón y avisa si la lámina no se había asentado;
   - grafica el mapa subiendo y bajando, la ganancia local $K(u)$ y el ensayo en el tiempo;
   - estima u_{mov}, la histéresis máxima y el u para 30°, 45° y 60°;
   - guarda todo en `mapa_MPAxx_fecha.mat`. **[ficha]**

Nombres que el guion espera del modelo

From Workspace: `u_prog`. **To Workspace** (formato *Array*): `y_deg`, `u_ap`, `t_ard` y, si existe, `latencia`. Si cambian un nombre en el modelo, cámbienlo también en el guion. El tiempo sale del **To Workspace** `t` del **Clock** o de `tout`; si no hay ninguno, el guion lo arma con el paso de 0.02 s. Si los datos duran menos que el ensayo programado, el guion se detiene: la corrida se cortó o las variables son de otra corrida.

7.4 Cómo se lee: el mapa de la MPA 4

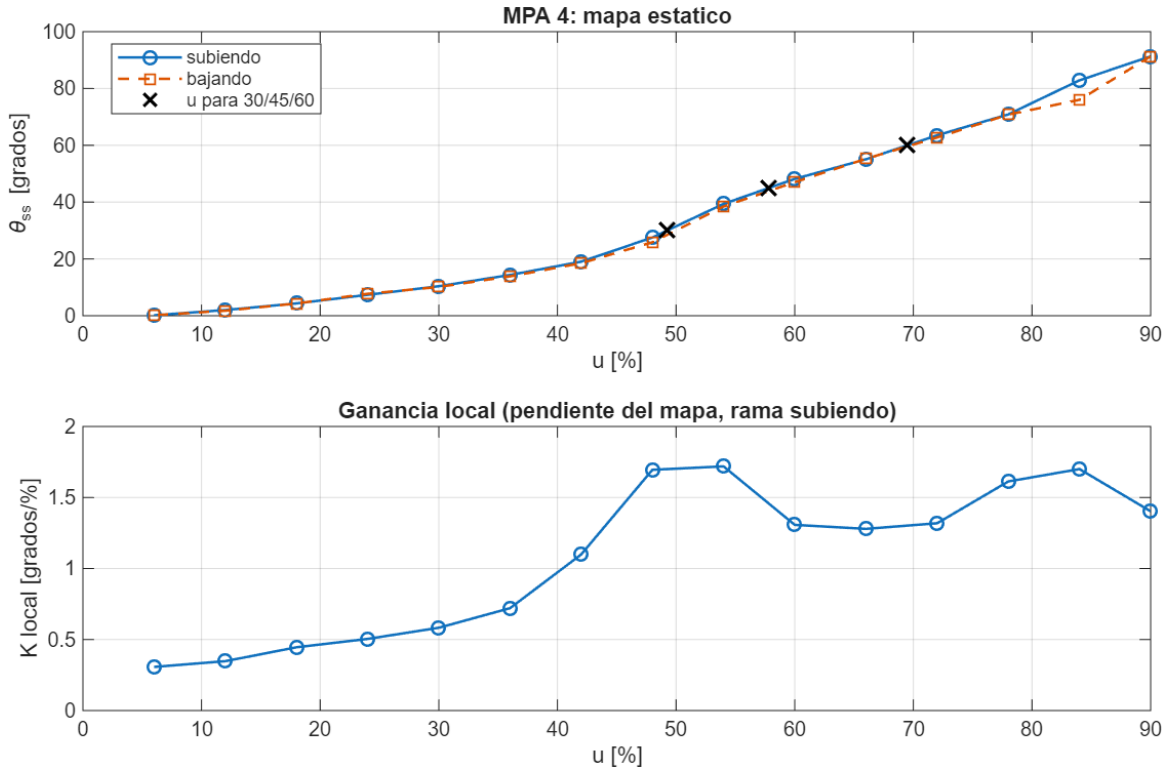


Figura 13: Mapa estático medido en la MPA 4 con `analisis_escalera.m` (escalones de 6 %). Arriba: θ_{ss} subiendo y bajando, con el u para 30°, 45° y 60°. Abajo: ganancia local $K(u) = d\theta_{ss}/du$ en la rama subiendo. Su MPA dará otros números, pero la forma debería parecerse.

Lo que dice este mapa de la planta

- **La curva tiene tres zonas.**
  - Por debajo de $u \approx 42\,\%$ la lámina sube poco: K va de 0.3 a 0.7 °/%. El empuje de una hélice crece más o menos con el cuadrado de su velocidad, así que con poco mando cada punto adicional aporta poco empuje.
  - Entre 48 y 75 % la curva es casi una recta, con $K \approx 1.3$–1.7 °/%. Es la mejor zona para operar.
  - Por encima de unos 78 % ($\theta > 70$) la lámina no se queda quieta: el chorro se vuelve turbulento, ahí están los escalones que no se asentaron y la única histéresis apreciable (unos 7° en 84 %). Esa zona no sirve como punto de operación.
- **Histéresis casi nula hasta 78 %**: las dos ramas se superponen. La fricción del eje no pesa en el modelo, y al punto de operación se puede llegar desde cualquier lado.
- **Despegue**: con 6 % la lámina sigue en reposo; con 12 % ya se separó unos 2°.
- **Mando para 30°, 45° y 60°**: 49.2, 57.8 y 69.5 %. Van a la ficha.

- **Punto de operación**: el reto asigna un ángulo θ_0 lejos de los dos topes. Del mapa sale el u_0 que lo produce. En la MPA 4, el tramo recto entre 45° y 60° (u_0 de 58 a 70 %, $K \approx 1.3$) es el más cómodo: ahí un escalón de 4 % mueve la lámina unos 5°, justo lo que pide la identificación.

- **El modelo lineal vale solo cerca de** u_0: entre 15° y 50° la ganancia local casi se duplica, y B es proporcional a K. Un controlador diseñado en un punto ve otra planta en el otro: puede quedar lento en uno y oscilar en el otro. Para recorrer toda la carrera hace falta cambiar las ganancias según el punto (*gain scheduling*).
- **Ganancia local**: la pendiente en u_0, $K_{\text{mapa}} \approx \big(\theta_{ss}(u_0+\Delta)-\theta_{ss}(u_0-\Delta)\big)/(2\Delta)$ [°/%]. Compárenla con la del escalón de la sección 8.4; si difieren más del 25 %, algo está mal en uno de los dos ensayos.
- **Histéresis**: si en su MPA subiendo y bajando difieren más de 3° en el mismo u, lleguen siempre al punto de operación desde el mismo lado y díganlo en el reporte.

8. Identificación para espacio de estados

8.1 La estructura del modelo

Dividiendo (2) por J queda un sistema de segundo orden,

$$\ddot{\tilde{\theta}} + 2\zeta\omega_n \dot{\tilde{\theta}} + \omega_n^2 \tilde{\theta} = K\omega_n^2 \tilde{u}, \qquad \omega_n^2 = \frac{k}{J}, \quad 2\zeta\omega_n = \frac{b}{J}, \quad K = \frac{\kappa}{k}\ [°/\%]. \tag{4}$$

Con estados físicos, $x_1 = \tilde{\theta}$ (ángulo) y $x_2 = \dot{\tilde{\theta}}$ (velocidad angular):

$$\dot{x} = \underbrace{\begin{bmatrix} 0 & 1 \\ -\omega_n^2 & -2\zeta\omega_n \end{bmatrix}}_{A} x + \underbrace{\begin{bmatrix} 0 \\ K\omega_n^2 \end{bmatrix}}_{B} \tilde{u}, \qquad \tilde{y} = \underbrace{\begin{bmatrix} 1 & 0 \end{bmatrix}}_{C} x, \qquad D = 0. \tag{5}$$

Las unidades: x_1 en °, x_2 en °/s, B en °/(%·s²). Todo el modelo cabe en **tres números por punto de operación**: ω_n, ζ y K. Los valores propios de A son $-\sigma \pm j\omega_d$, con $\sigma = \zeta\omega_n$ y $\omega_d = \omega_n\sqrt{1-\zeta^2}$.

La idea: un ensayo para cada matriz

- A **sale de la lámina sola.** Se aplica un pulso corto de mando; cuando termina, el mando queda constante en u_0 y la lámina oscila libre alrededor de θ_0. Esa *respuesta libre* depende solo de A: la separación entre picos da ω_d y la envolvente da σ. Son los polos, medidos directamente.
- B **sale del motor.** Un escalón pequeño Δu lleva la lámina a otro equilibrio: $K = \Delta\theta_{ss}/\Delta u$, y con eso B.
- C **ya se conoce**: se mide el ángulo y no la velocidad.

8.2 El ensayo programado

Todo sale de **una corrida de unos 5 minutos** con el modelo de la figura 12: armado, llegada al punto de operación, tres pulsos (para promediar A), un escalón hacia arriba (para estimar K) y uno hacia abajo (para *validar*, sin usarlo para estimar).

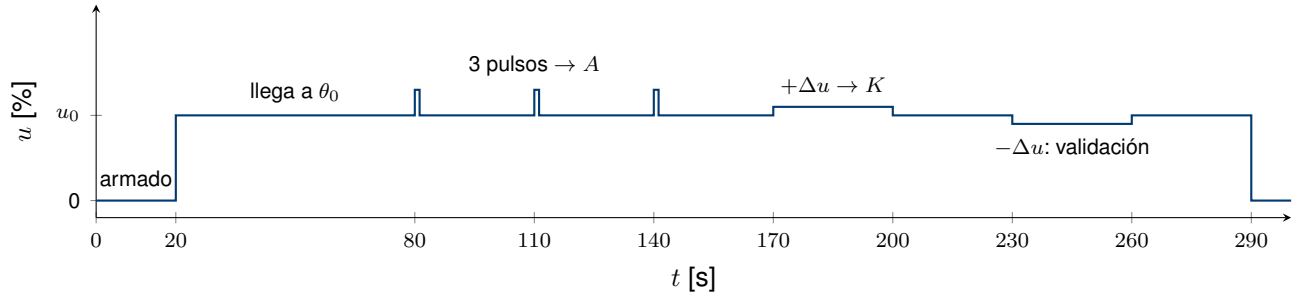


Figura 14: Forma del programa (el ancho de los pulsos está exagerado para que se vean). El firmware redondea los bordes con su rampa.

Código 3: Programa del ensayo de identificación (`programa_identificacion.m`).

```
modulo = 4;    %       numero de SU MPA
u0     = 40;   % [%]   mando del punto de operacion (del mapa estatico)
Ap     = 10;   % [%]   amplitud del pulso
tp     = 0.4;  % [s]   duracion del pulso
dU     = 4;    % [%]   escalon pequeno
Tarm = 20;     % [s]   u = 0 para armar el ESC (encender la fuente aqui)
Tesp = 30;     % [s]   espera entre eventos
rampa = 50;    % [%/s] RAMPA_PCT_S del firmware
ev = [0 0; Tarm u0];
t  = Tarm + 2*Tesp;                          % 60 s para asentarse en theta0
tPul = zeros(1,3);
for k = 1:3
    tPul(k) = t;
    ev = [ev; t u0+Ap; t+tp u0];   t = t + Tesp;
end
tSub = t;  ev = [ev; t u0+dU];    t = t + Tesp;    % escalon arriba (estimacion)
           ev = [ev; t u0];       t = t + Tesp;    % regreso
tBaj = t;  ev = [ev; t u0-dU];    t = t + Tesp;    % escalon abajo (validacion)
           ev = [ev; t u0];       t = t + Tesp;
ev = [ev; t 0];                                   % fin: motor a cero
u_prog = timeseries(ev(:,2), ev(:,1));
Tfin = t + 5;                                     % Stop time
```

Cómo escoger A_p, t_p y Δu

- El pulso debe sacar la lámina entre **8° y 15°** en el primer pico. Con la rampa, un pulso de $A_p = 10\,\%$ tarda $0.2\,\mathrm{s}$ en subir y $0.2\,\mathrm{s}$ en bajar: t_p debe ser mayor que $A_p/50$ s.
- El escalón Δu debe mover el equilibrio entre **5° y 8°**: $\Delta u \approx 6/K_{\text{mapa}}$. Con 4 % la rampa tarda $0.08\,\mathrm{s}$: despreciable frente a la lámina.
- Si al llegar el siguiente evento la lámina todavía oscila, aumenten `Tesp`.

Después de la corrida comprueben la latencia y `diff(t_ard)`. Las variables de los **To Workspace** se sobrescriben en el siguiente *Run*; por eso `analisis_pulsos.m` guarda primero la corrida completa

(`y_deg`, `u_ap`, `t_ard` y el programa del ensayo) en `ident_MPAxx_fecha.mat`. Para retomar otro día basta con `load` de ese archivo.

8.3 La matriz A: respuesta libre después del pulso

Cuando termina la rampa de bajada del pulso ($t_{\text{pulso}} + t_p + A_p/50$), el mando queda fijo en u_0 y la lámina se mueve sola. Con $\tilde{u} = 0$, la solución de (4) es una cosenoide amortiguada que depende solo de A:

$$\tilde{\theta}(t) = A_m\, e^{-\sigma t}\cos(\omega_d t + \varphi) + c, \qquad \omega_n = \sqrt{\sigma^2 + \omega_d^2}, \qquad \zeta = \frac{\sigma}{\omega_n}. \tag{6}$$

`analisis_pulsos.m` ajusta los cinco parámetros (A_m, σ, ω_d, φ y un pequeño desfase c) a **todos** los puntos de los 6 s de respuesta libre, con `fminsearch` y varios valores iniciales de ω_d. Así no depende de cuántos picos se alcancen a ver: en la MPA 4, con $\zeta \approx 0.3$, la lámina da solo dos extremos por encima del ruido.

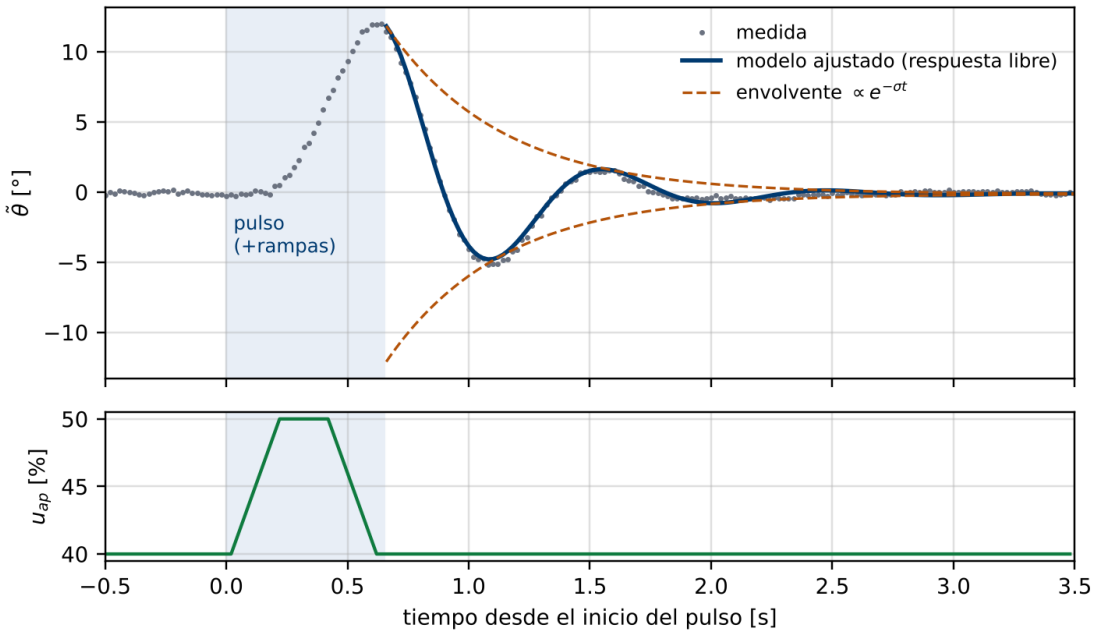


Figura 15: Primer pulso del ensayo de la MPA 4 ($u_0 = 40\,\%$, $\theta_0 \approx 17$, $A_p = 10\,\%$, $t_p = 0.4$ s). El modelo (6) se ajusta desde que termina la rampa de bajada: $\sigma = 2.10$ s^{-1}, $\omega_d = 6.78$ rad/s, es decir $\omega_n = 7.10$ rad/s y $\zeta = 0.30$. Note el retardo: la lámina empieza a moverse unos 0.1 s después del mando.

Lectura a ojo, para comprobar el ajuste

Dos extremos sucesivos están separados medio periodo, π/ω_d, y su cociente es $e^{-\sigma\pi/\omega_d}$. En la figura 15: mínimo de -4.9 en 1.08 s y máximo de $+1.6$ en 1.55 s. Entonces $\omega_d \approx \pi/0.47 \approx 6.7$ rad/s y $\sigma \approx \ln(4.9/1.6)/0.47 \approx 2.4$ s^{-1}, cerca del ajuste. Con solo dos extremos la lectura a ojo es frágil; el ajuste usa los 300 puntos.

Código 4: Núcleo de `analisis_pulsos.m` (extracto; el archivo también grafica y avisa).

```
Tlib = 6;                                   % [s] de respuesta libre
opc  = optimset('Display', 'off', 'MaxFunEvals', 5000, 'MaxIter', 5000);
for k = 1:numel(tPul)
    y0 = mean(y(t >= tPul(k)-5 & t < tPul(k)));      % equilibrio antes del pulso
    tl = tPul(k) + tp + Ap/rampa + 0.05;             % termino la rampa de bajada
    w  = t >= tl & t < tl + Tlib;                    % respuesta libre
    tt = t(w) - tl;   z = y(w) - y0;
    libre = @(p) p(1)*exp(-p(2)*tt).*cos(p(3)*tt + p(4)) + p(5);
    J     = @(p) sum((z - libre(p)).^2);
    Jmin = inf;
    for wd0 = [3 5 7 9 12]                           % varios arranques en wd
        [p, Jp] = fminsearch(J, [z(1) 2 wd0 0 0], opc);
        if Jp < Jmin, Jmin = Jp; pb = p; end
    end
    res(k,:) = [pb(2), abs(pb(3)), 100*(1 - sqrt(Jmin)/norm(z - mean(z))), max(abs(z))];
end
sigma = mean(res(:,1));  wd = mean(res(:,2));
wn = hypot(sigma, wd);   zeta = sigma/wn;
```

Cuándo creerle al resultado

Los tres pulsos deben dar ω_n dentro de $\pm 10\,\%$ y ζ dentro de $\pm 25\,\%$ entre sí, y cada ajuste un *fit* por encima del 70 %. En la MPA 4 dieron $\omega_n = 7.10$, 6.15 y 6.17 rad/s y $\zeta = 0.30$, 0.35 y 0.32, con *fit* de 88–89 %: promedio $\omega_n = 6.47$ rad/s, $\zeta = 0.32$. Si un pulso se sale, miren su gráfica: un golpe contra el tope, una racha del chorro o un pulso que no alcanzó a asentarse lo explican casi siempre.

8.4 La matriz B: los escalones

Código 5: Ganancia estática con el escalón de subida (`analisis_escalones_modelo.m`, parte 1).

```
ym  = @(a,b) mean(y(t >= a & t < b));                    % promedio en [a, b)
Ksub = (ym(tSub+Tesp-5, tSub+Tesp) - ym(tSub-5, tSub)) / dU;
Kbaj = (ym(tBaj+Tesp-5, tBaj+Tesp) - ym(tBaj-5, tBaj)) / (-dU);
fprintf('K subiendo = %.3f   K bajando = %.3f  grados/%%\n', Ksub, Kbaj)
```

Si K_{sub} y K_{baj} difieren más del 15 %, el escalón es demasiado grande para la curvatura del mapa en ese punto, o la histéresis está pesando: repitan con Δu menor.

8.5 Las matrices, el retardo y la validación

Código 6: Modelo en espacio de estados y validación con el escalón de bajada (parte 2).

```
K = Ksub;
A = [0 1; -wn^2 -2*zeta*wn];   B = [0; K*wn^2];   C = [1 0];   D = 0;
G = ss(A, B, C, D, 'StateName', {'theta','omega'}, 'StateUnit', {'grados','grados/s'}, ...
       'InputName', 'u', 'InputUnit', '%', 'OutputName', 'theta', 'OutputUnit', 'grados');
% --- datos de validacion: escalon de bajada, en desviacion (u aplicado de verdad)
u  = double(u_ap(:));
w  = t >= tBaj-5 & t < tBaj+Tesp;
tv = t(w) - t(find(w,1));
dy = y(w) - ym(tBaj-5, tBaj);    du = u(w) - u0;
% --- retardo: el que mejor alinea modelo y medida
dl = 0:Ts:0.3;   J = zeros(size(dl));
for i = 1:numel(dl)
    G.InputDelay = dl(i);   J(i) = norm(dy - lsim(G, du, tv));
end
[~, ib] = min(J);  retardo = dl(ib);  G.InputDelay = retardo;
ys  = lsim(G, du, tv);
fit = 100*(1 - norm(dy - ys)/norm(dy - mean(dy)));
plot(tv, dy, tv, ys, '--', 'LineWidth', 1.2); grid on
legend('medido', 'modelo'); xlabel('t [s]'); ylabel('\Delta\theta [grados]')
fprintf('retardo = %.2f s   fit de validacion = %.0f %%\n', retardo, fit)
save(sprintf('MPA%02d_modelo_%s.mat', modulo, datestr(now,'yyyymmdd')), ...
     'A','B','C','D','wn','zeta','K','u0','retardo','fit')
```

El retardo que aparece junta el viaje por el USB, el pulso de $20\,\mathrm{ms}$ y la dinámica del motor que el modelo de segundo orden deja por fuera. **No va en las matrices**: queda anotado y entra al lazo simulado, porque es lo que puede volver oscilante un diseño demasiado rápido. La validación usa `u_ap`, el mando que de verdad aplicó el Arduino.

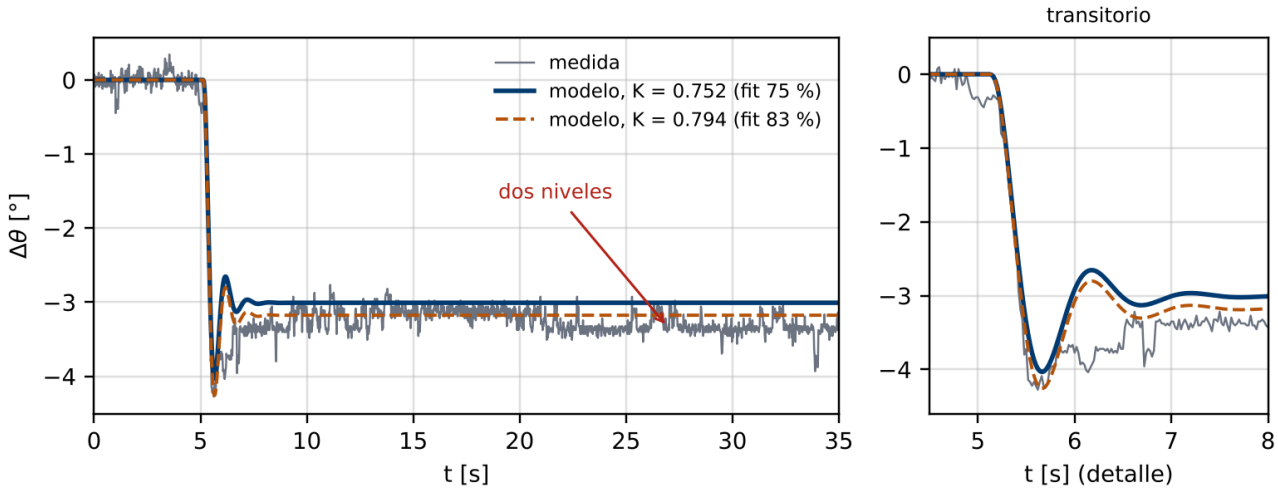


Figura 16: Validación con el escalón de bajada en la MPA 4 ($u_0 = 40\,\%$, $\Delta u = -4\,\%$), que no se usó para estimar. Modelo de los pulsos ($\omega_n = 6.47$ rad/s, $\zeta = 0.32$) con retardo de 0.10 s. Continua: K del escalón de subida; discontinua: promedio de subida y bajada.

Lo que dice esta validación

- **La caída se reproduce bien**: el instante en que arranca (retardo), la pendiente y el primer mínimo coinciden. Esa es la parte que gobierna un lazo cerrado.

- **El nivel final difiere unos 0.3°**: bajando, la planta respondió con $K = 0.837$ y subiendo con 0.752, un 11 % menos. Es la curvatura del mapa estático alrededor de u_0. Con el promedio, 0.794, el *fit* pasa de 75 a 83 %. La acción integral del Reto 3 elimina ese error en estado estacionario, pero el diseño debe tolerarlo.
- **Bajando, la lámina se amortigua más**: después del mínimo casi no rebota, y el modelo sí. La planta no es simétrica.
- **La medida salta entre dos niveles**, unos -3.1 y -3.35, y se queda un rato en cada uno: no es ruido blanco. Puede ser el chorro alternando entre dos patrones de flujo o la resolución del potenciómetro (unos 0.25° por cuenta). Ningún modelo lineal lo reproduce; para el control es una perturbación.

Modelo de la MPA 4 en $\theta_0 \approx 17.4$, $u_0 = 40\,\%$

$$A = \begin{bmatrix} 0 & 1 \\ -41.9 & -4.15 \end{bmatrix}, \qquad B = \begin{bmatrix} 0 \\ 31.5 \end{bmatrix}, \qquad C = \begin{bmatrix} 1 & 0 \end{bmatrix}, \qquad \lambda(A) = -2.08 \pm 6.13j, \qquad T_d \approx 0.10 \text{ s}.$$

Con $\omega_n = 6.47$ rad/s, $\zeta = 0.32$ y $K = 0.752$ °/% (escalón de subida). Es el ejemplo de este manual; cada equipo identifica su MPA en su propio punto de operación.

Preguntas de validación (no hay un *fit* mágico)

- ¿El modelo reproduce el escalón de bajada, que no se usó para estimar: sentido, valor final ($\pm 15\,\%$), número de oscilaciones visibles y periodo?
- ¿K del escalón y K_{mapa} de la escalera concuerdan ($\pm 25\,\%$)?
- ¿El periodo de las oscilaciones del escalón coincide con el ω_d de los pulsos?
- ¿El modelo se identificó en el mismo punto de operación donde se va a controlar?

Con eso, anoten ω_n, ζ, K, el retardo, u_0 y θ_0. **[ficha]**

8.6 Otra vía: el escalón con `ident` (opcional)

El mismo modelo con lo que vieron en Modelos y Simulación

El escalón de subida, recortado y trasladado al origen, se puede llevar a `ident` con un *Process Model* de 2 polos subamortiguado con retardo (P2DU). Da K, ω_n, ζ y T_d de una vez, y con ellos se arman A, B y C con (5). En la MPA 4 dio $K = 0.82$ °/%, $\omega_n = 5.9$ rad/s, $\zeta = 0.28$ y $T_d = 0.08$ s, en línea con los pulsos ($\omega_n = 6.47$, $\zeta = 0.32$). Que dos ensayos distintos den los mismos polos es la mejor prueba de que el modelo es creíble. Es también el camino si la lámina casi no oscila después del pulso.

Si la respuesta libre casi no se separa del ruido (lámina muy amortiguada) o el ajuste da un *fit* bajo, primero intenten un pulso un poco mayor, sin pasar de 15°. Por línea de comandos, el equivalente de `ident` es:

Código 7: Alternativa: dos polos, sin ceros, con retardo estimado.

```
w   = t >= tSub-5 & t < tSub+Tesp;
dat = iddata(y(w) - ym(tSub-5,tSub), u(w) - u0, Ts);
G2  = tfest(dat, 2, 0, NaN);            % 2 polos, 0 ceros, retardo libre
[num, den] = tfdata(G2, 'v');           % den = [1  2*zeta*wn  wn^2]
wn = sqrt(den(3));  zeta = den(2)/(2*wn);  K = num(end)/den(3);
```

Las matrices se arman igual con (5). No usen la realización que entrega el toolbox (`ss(G2)` o `idss`) sin mirarla: sus estados no son ángulo y velocidad, y el Reto 3 los necesita con significado físico.

8.7 Lo que el modelo deja por fuera

Cuadro 6: Hipótesis del modelo y dónde se notan.

Lo que se ignora	Dónde aparece
Dinámica del motor y la hélice (τ_m)	Como retardo aparente en el escalón. Es el tercer estado que no se modela.
Retardo del USB y del pulso de $20\,\mathrm{ms}$	Igual que el anterior: se mide junto.
No linealidad de la planta	K, ω_n y ζ cambian con θ_0. El modelo vale cerca de su punto.
Rampa del firmware	Limitador de rapidez de ± 50 %/s. Inactivo en señal pequeña; activo en cambios grandes de referencia.
Zona muerta, saturación y tope	El mando útil está en $[u_{\text{arr}},\ 100]$; por debajo de u_{mov} la lámina descansa en el tope.
Empuje en un solo sentido	La hélice solo empuja hacia arriba; hacia abajo actúa la gravedad.
Cuantización del sensor	Del orden de 0.2 °/cuenta (menos con el promedio de 16). Al derivar y con $T_s = 0.02\,\mathrm{s}$, una sola cuenta equivale a unos **10 °/s** de «velocidad»: la velocidad angular no se mide, se estima.
Turbulencia del chorro	Pequeñas oscilaciones irregulares sobre el equilibrio que ningún modelo lineal reproduce.

8.8 Otro punto de operación (si el reto lo pide)

Repitan el ensayo programado con otro θ_0 (por ejemplo 30° y 60°). Comparar ω_n, ζ y K entre los dos puntos muestra, con números, cuánto cambia la planta, y es la base de la programación de ganancias (*gain scheduling*).

9. Preparar el lazo cerrado

El controlador y el observador se diseñan en el Reto 3. Este manual deja lista la **interfaz**: cómo entran y salen las señales, en qué unidades y qué se registra.

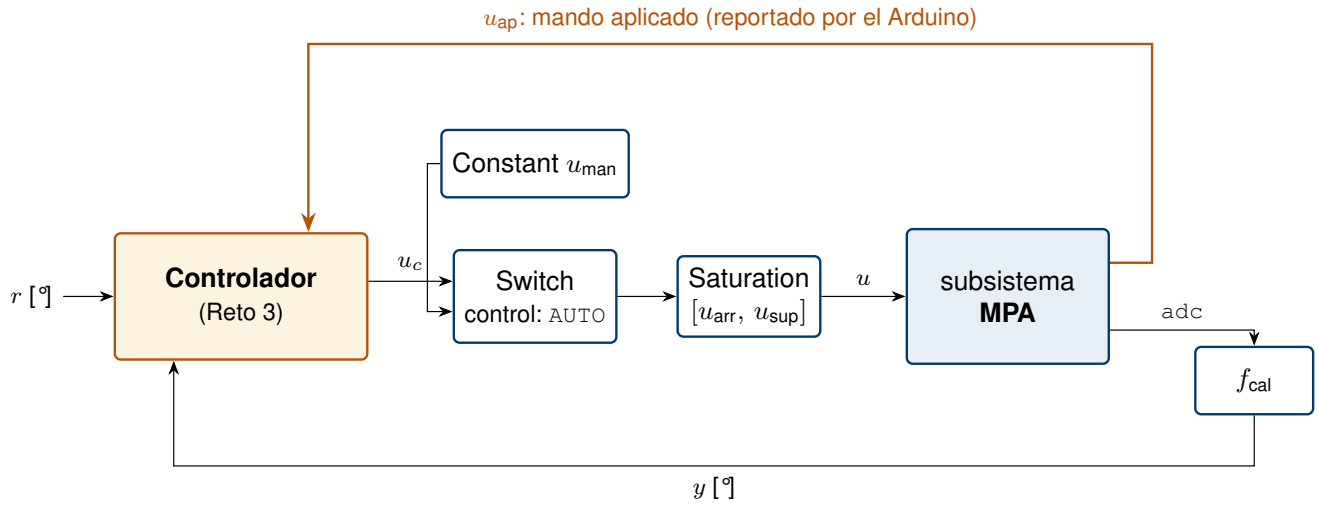


Figura 17: Interfaz de `MPA_XX_lazo.slx`. Todo lo que está fuera de la caja ámbar queda fijo. u_{ap} viene del Arduino: ya incluye la rampa, el tope y el estado SEGURO.

- **Unidades absolutas afuera, desviaciones adentro.** La caja recibe r e y en grados y u_{ap} en %, y entrega u_c en %. Si el diseño trabaja en desviación, la caja resta θ_0 y u_0 al entrar y suma u_0 al salir.
- **La caja recibe el mando aplicado.** El controlador del Reto 3 tiene estados internos, y esos estados necesitan saber qué recibió de verdad la planta: en manual no es u_c; cuando el mando satura o la rampa lo frena, tampoco. Es la misma idea del seguimiento del Reto 2, y aquí la señal la mide el propio Arduino.
- **La saturación usa los límites de la ficha**, $[u_{arr}, u_{sup}]$.
- **El arranque no cambia**: MAN con $u_{man} = 0$, fuente encendida, armado, $u_{man} = u_0$, esperar a que la lámina se asiente en θ_0, y solo entonces AUTO.
- **Registro**: r, y, u_c, u_{ap}, `AUTO`, `t_ard` y la latencia, con **To Workspace** a T_s.
- **Lazo simulado en paralelo**: **Saturation** y **Rate Limiter** (± 50 %/s) seguidos de un **State-Space** con las matrices identificadas y el retardo, alimentado con $u - u_0$ y con θ_0 sumado a su salida. Arranca en equilibrio y permite comparar planta y modelo con los mismos mandos.

10. Problemas frecuentes

Cuadro 7: Síntoma, comprobación y acción.

Síntoma	Comprueben	Acción
Simulink se queda esperando y sale un *timeout*	Puerto COM, monitor serie del IDE abierto, firmware cargado	Cierren el IDE, revisen el COM; si alguien usó Connected IO, vuelvan a cargar el firmware.
Error de tamaños en una MATLAB Function	Se pegó el código completo, con sus funciones auxiliares	Peguen el archivo entero; *Data type* del **Serial Receive** en `uint8`.
`valido` en 0, `n_malos` sube	Versiones distintas de firmware y funciones	Firmware v2 con los `.m` de la misma versión.
El ESC pita y no arma	¿La fuente se encendió con el modelo corriendo en $u = 0$? Señal en pin 6 y tierra común	Apaguen la fuente, 5 s, enciendan con el modelo en cero.
`estado` en 0 y el motor no arranca	¿Se arrancó con $u \neq 0$? ¿`habilitar` en 0?	Lleven u a 0 (rearme) y `habilitar` a 1.
El motor gira pero la lámina casi no sube	Hélice volteada o giro al revés	Sección 4: orientación y, si hace falta, intercambiar dos fases.
La fuente se protege (se apaga)	Cambios bruscos, `RAMPA_PCT_S` en 0	Desconecten la fuente de la red unos segundos; mantengan la rampa.
Ángulo incorrecto	Calibración de este módulo	Comprueben tres marcas con el motor apagado; recalibren.
Latencia mayor que 1	Scopes o Displays abiertos, otros programas	Cierren ventanas; registren con **To Workspace**.
El ensayo tarda el doble; `diff(t_ard)` da 0.04 y `diff(seq)` da 2	*Block sample time* del **Serial Receive** y *Fixed-step size* del modelo	Los dos en 0.02. Con 0.01, Simulink lee dos paquetes por paso y su tiempo avanza a la mitad del real: el mapa estático no cambia, pero la identificación da ω_n a la mitad.
Ajuste del pulso con *fit* bajo o ω_n dispersa	Gráfica de cada pulso: tamaño del primer pico, golpes, rachas	Pulso algo mayor (A_p o t_p); si no, sección 8.6.
K subiendo $\neq K$ bajando	Tamaño de Δu, histéresis	Reduzcan Δu; lleguen al punto desde el mismo lado.

11. Lista de comprobación

Antes de encender la fuente

☐ Módulo identificado y ficha a la mano. ☐ Firmware v2 cargado; monitor serie cerrado.
☐ COM correcto en los tres bloques seriales. ☐ Calibración de *este* módulo cargada.
☐ $u = 0$ y `habilitar` $= 1$. ☐ Simulink corriendo: `estado` $= 1$, `valido` $= 1$, latencia 0–1.
☐ Hélice orientada y sentido del aire comprobado. ☐ Manos lejos de la hélice.

Antes de aceptar un modelo

☐ `diff(t_ard)` $= 0.02$ y latencia 0–1 en todo el ensayo. ☐ Lámina sin tocar topes.
☐ Tres pulsos coherentes (ω_n $\pm 10\,\%$, ζ $\pm 25\,\%$). ☐ K_{sub} y K_{baj} dentro del 15 %.
☐ K del escalón y del mapa dentro del 25 %. ☐ Validación con el escalón de bajada.
☐ Matrices con estados físicos y unidades. ☐ Ficha actualizada y `.mat` guardados con fecha.

A. Ficha del módulo

Una por unidad. Se imprime, se llena a mano la primera vez y se guarda con el módulo. Si ya existe, comprueben al menos la calibración en tres marcas, u_{mov} y u para 45° antes de usarla.

Módulo MPA N.º		**Fecha / pareja**	
Tarjeta (UNO / Leonardo)		**Puerto COM / pin ESC**	
Firmware: versión, `PULSO_100`, `PULSO_TOPE`			
Hélice: tamaño y aire hacia la lámina comprobado		**n al subir la lámina**	aumenta disminuye
Sensor			
Calibración: tipo y coeficientes			
Error máximo / de validación [°]		**Resolución [°/cuenta]**	
Mando			
u_{arr}: gira la hélice [%]		**u_{mov}: se despega [%]**	
u_{85} / u_{sup} [%]		**Histéresis máxima [°]**	
u para 30° / 45° / 60° [%]			
Modelo lineal			
Punto de operación (u_0, θ_0)		**K_{mapa} [°/%]**	
ω_n [rad/s] / ζ		**K del escalón [°/%]**	
Retardo aparente [s]		***Fit* de validación [%]**	
Observaciones			

B. Hojas de campo

Calibración del sensor (motor apagado)

θ [°]	`adc` **subiendo**	`adc` **bajando**
0		
10		
20		
30		
40		
50		
60		
70		
80		
90		
Validación (no se usan en el ajuste)		
15		
45		
75		

Primera prueba con hélice

Aire hacia la lámina con $u \approx 20\,\%$	
u_{arr} **(gira la hélice)**	
u_{mov} **(se despega la lámina)**	
u_{85} **(lámina en** $\approx$**85°)**	
¿Se protegió la fuente? ¿Cuándo?	

Ensayo programado

u_0 / θ_0	
A_p, t_p, Δu, `Tesp`	
Primer pico del pulso [°]	
Latencia máxima	
Archivo `.mat`	

Procedencia

Este manual reúne y reemplaza: (1) C. Barco Díaz y M. Sánchez Monsalve, *Módulo de control de posición angular: manual de uso*, EIA, 2016 (descripción original del módulo); (2) S. A. Castaño Giraldo, *Guía de implementación en Simulink del MPA* y *Guía de calibración del sensor e identificación de sistemas*, 2026-1 (curva de calibración de ejemplo). El enlace serial sigue el esquema usado en 2026-1 con la bomba de infusión del curso de Biomecatrónica. Las fotografías son de la MPA 4 (2026-2). La figura 3 es un esquema; el mapa estático (figura 13) y la respuesta al pulso (figura 15) se midieron en la MPA 4 el 6 de octubre de 2026. Firmware, funciones y guiones: repositorio del curso (archivo `README.md`).